

ModelFaucet

白皮书：把任意软件变成 AI-LLMs 最后一公里分销渠道的开源基础设施

ModelFaucet Project

2026-06-17

- [ModelFaucet 白皮书](#)
 - [目录](#)
 - [1. 项目命名与定位](#)
 - [1.1 推荐名称](#)
 - [1.2 为什么叫 ModelFaucet](#)
 - [1.3 项目范畴](#)
 - [1.4 非目标](#)
 - [2. 背景与问题](#)
 - [3. 市场已有方案与空白](#)
 - [4. 设计原则](#)
 - [4.1 End user 默认无感](#)
 - [4.2 高级用户可自带酒水](#)
 - [4.3 隐私用户可走本地模型](#)
 - [4.4 渠道收益自动化](#)
 - [4.5 真实密钥永远不进客户端](#)
 - [4.6 OpenAI-compatible 优先](#)
 - [5. 核心产品形态](#)
 - [5.1 Faucet SDK](#)
 - [5.2 Faucet Gateway](#)
 - [5.3 Developer Console](#)
 - [5.4 End-user Settings](#)
 - [5.5 Local Bridge](#)
 - [6. 四种使用模式](#)
 - [6.1 模式 A：平台默认水龙头](#)
 - [6.2 模式 B：开发者自带线路](#)
 - [6.3 模式 C：End user BYOK](#)
 - [6.4 模式 D：本地 / 局域网模型](#)
 - [7. 总体架构](#)
 - [8. 核心模块](#)
 - [8.1 Session Broker](#)
 - [8.2 Gateway Router](#)
 - [8.3 Key Vault](#)

- 8.4 Usage Ledger
- 8.5 Rating Engine
- 8.6 Settlement Engine
- 9. 商业与分账模型
 - 9.1 平台默认模式
 - 9.2 开发者自带线路模式
 - 9.3 BYOK 模式
 - 9.4 本地模型模式
- 10. 安全、合规与隐私边界
 - 10.1 不在客户端暴露真实 provider key
 - 10.2 不让云端访问用户内网
 - 10.3 用户数据最小化
 - 10.4 PII 策略
 - 10.5 合规提示
- 11. 技术实施方案
 - 11.1 推荐技术栈
 - 11.2 Monorepo 结构
 - 11.3 MVP 数据库表
 - 11.4 MVP API
- 12. MVP 路线图
 - 阶段 0: 规格冻结
 - 阶段 1: 最小闭环
 - 阶段 2: BYOK
 - 阶段 3: Local Bridge
 - 阶段 4: 开发者自带线路与分账
 - 阶段 5: 开源发布
- 13. 开源策略
- 14. 风险与应对
 - 14.1 API key 泄露风险
 - 14.2 上游 provider 政策风险
 - 14.3 小额高频结算复杂
 - 14.4 本地模型质量不稳定
 - 14.5 渠道作弊和薅羊毛
- 15. 参考资料
- 结语

ModelFaucet 白皮书

中文名：模型水龙头

英文名：ModelFaucet

一句话定位：Open-source LLM distribution gateway for every app.

ModelFaucet 是一套开源的 AI-LLMs 集成与分销基础设施。它让任何网站、App、桌面软件、插件、企业系统，只要嵌入 SDK，就能为 end users 提供与当前软件场景高度匹配的 AI 功能，同时让软件开发者自动获得 Token 流动带来的渠道收益。

它不是单纯的聊天 SDK，也不是单纯的 LLM Gateway。它是一套由 SDK、Gateway、Key Vault、Usage Ledger、Rating Engine、Settlement、Local Bridge 组成的“AI 分销协议层”。

目标：让每一款软件都成为 AI-LLMs to end users 的最后一公里分销渠道。

目录

1. 项目命名与定位
 2. 背景与问题
 3. 市场已有方案与空白
 4. 设计原则
 5. 核心产品形态
 6. 四种使用模式
 7. 总体架构
 8. 核心模块
 9. 商业与分账模型
 10. 安全、合规与隐私边界
 11. 技术实施方案
 12. MVP 路线图
 13. 开源策略
 14. 风险与应对
 15. 参考资料
-

1. 项目命名与定位

1.1 推荐名称

ModelFaucet

中文名：**模型水龙头**

口号：**Turn every app into an AI last-mile channel.**

中文口号：**让每一款软件都成为 AI 的最后一公里渠道。**

1.2 为什么叫 ModelFaucet

“Faucet” 是水龙头，代表一种对终端用户而言“打开即可流出”的体验。用户不需要知道水来自哪里，也不需要理解管道、泵站、计费表和分成系统。ModelFaucet 把大模型 API、渠道归因、用量计费、自动分润、本地模型、BYOK 这些复杂基础设施封装成一个可嵌入的开源模块。

1.3 项目范畴

ModelFaucet 覆盖五个层次：

- | | |
|----------------------|------------------|
| 1. Embeddable SDK | 嵌入任意软件的前端/后端 SDK |
| 2. LLM Gateway | 多模型统一入口与路由层 |
| 3. Key Vault | 平台、开发者、用户密钥托管 |
| 4. Usage Ledger | Token 用量事实账本 |
| 5. Settlement Engine | 自动定价、利润计算与渠道分账 |

1.4 非目标

ModelFaucet 第一阶段不做以下事情：

- 不做通用大模型 App 商店
- 不做完整 Agent 市场
- 不做向量数据库/RAG 平台的全栈替代品
- 不做模型训练、微调、GPU 云租赁平台
- 不把真实 provider API key 写入开源 SDK 或客户端

2. 背景与问题

LLM 的能力正在从独立聊天产品向垂直软件内部迁移。CRM 需要自动回复客户，文档工具需要总结和改写，电商后台需要生成商品描述，客服系统需要工单归纳，表格软件需要自动分析数据，低代码平台需要自然语言生成流程。

但是当前大多数 AI 集成方式仍然存在明显摩擦：

用户侧摩擦：

- 小白用户不懂 API key
- 用户不知道该选哪个模型
- 用户不知道如何充值、限额、处理账单
- 用户担心隐私，不知道什么时候应该用本地模型

开发者侧摩擦：

- 每个 provider API 不同
- 需要自己做 key 管理、限流、重试、fallback
- 需要自己做 token 计量、成本核算、用户账单
- 想通过 AI 功能赚钱，但分账和提现很复杂

平台侧摩擦：

- LLM 成本随模型、provider、缓存、上下文长度变化
- 转售和 BYOK 的边界不清
- 把真实 key 下发到客户端会导致泄露和滥用

ModelFaucet 的机会在于：把“LLM 使用能力”和“Token 商业流动能力”同时产品化。

3. 市场已有方案与空白

已有产品可以分为四类：

类别	代表方案	已解决的问题	仍然缺口
开源 LLM Gateway	LiteLLM、Bifrost、Portkey 等	多模型统一接口、virtual keys、成本追踪、fallback	主要面向企业或开发者，不是“任何软件自动变渠道”的分销层
AI Gateway SaaS	Cloudflare AI Gateway、Vercel AI Gateway、ngrok AI Gateway	统一 endpoint、密钥托管、预算、观测、BYOK	不是开源分销协议，也不以内嵌式渠道分润为核心
模型聚合市场	OpenRouter 等	一个 key 访问多模型，聚合模型目录和账单	分销归因、渠道钱包、自动结算不是核心
Token Billing	Stripe LLM token billing 等	token 计量、模型价格同步、markup、账单	仍需与具体 SDK、Gateway、渠道系统集成

最重要的市场空白是：

现有系统解决了“怎么调用模型”，但没有完整解决“怎么让任何软件变成 AI 分销渠道，并自动获得 Token 流动利润”。

ModelFaucet 的差异化不是再造一个模型网关，而是把模型网关能力放到一个分销协议中：

```
LLM Gateway
+ Embeddable vertical SDK
+ BYOK / Local model UX
+ Usage Ledger
+ Rating Engine
+ Channel attribution
+ Automatic payout
```

4. 设计原则

4.1 End user 默认无感

普通用户不应该被要求理解 API key、provider、模型价格、tokens、上下文窗口。默认模式下，用户只需要在当前软件里点击“AI 帮我完成”。

4.2 高级用户可自带酒水

高级用户应能填入自己的 OpenAI、Anthropic、Gemini、OpenRouter、Azure、Cloudflare、Vercel 或自定义 OpenAI-compatible endpoint。用户可以多填 key，配置优先级、预算和 fallback。

4.3 隐私用户可走本地模型

用户可以接入 Ollama、vLLM、LM Studio、llama.cpp server 或局域网内模型。敏感内容可以被强制路由到本地模型。

4.4 渠道收益自动化

软件开发者不应该每天手动核对 token、账单、成本和利润。每次请求都要自动形成 usage event，并自动计算渠道收益。

4.5 真实密钥永远不进客户端

开源 SDK 和客户端应用不得内置真实 provider API key。它们只能持有 public app id、feature manifest、短期 session token 或 virtual key。真实 provider key 只存在服务端 vault/KMS 中。

4.6 OpenAI-compatible 优先

对开发者而言，最小学习成本来自 OpenAI-compatible API。ModelFaucet 对外应优先暴露 `/v1/chat/completions`、`/v1/responses`、`/v1/embeddings` 等兼容接口，同时提供更高级的场景 API。

5. 核心产品形态

5.1 Faucet SDK

嵌入第三方软件的 SDK，提供：

- `createFaucet()` 初始化
- `chat()` / `generate()` / `stream()` 调用
- 垂直 feature 调用
- BYOK 设置组件
- Local model 设置组件
- credits 购买组件
- 用量展示组件

示例：

```
import { createFaucet } from "@modelfaucet/sdk";

const faucet = createFaucet({
  publicAppId: "app_pub_123",
  user: { id: currentUser.id }
});

const result = await faucet.chat({
  feature: "customer_reply",
  input: {
    ticket_text: "客户投诉物流慢，要求退款。",
    tone: "诚恳、专业、不承认法律责任"
  }
});
```

5.2 Faucet Gateway

统一 LLM Gateway，负责：

- session token 鉴权
- 模型路由
- provider key 选择
- 预算检查
- 限流
- fallback
- streaming proxy

- token usage 捕获
- usage event 写入

5.3 Developer Console

开发者后台，负责：

- 注册开发者
- 创建 app
- 获取 public_app_id
- 配置 feature manifest
- 配置开发者自己的 provider keys
- 设置 markup 和渠道分成比例
- 查看用量、收入、提现状态

5.4 End-user Settings

终端用户设置页，负责：

- 查看 credits 余额
- 充值 credits
- 填写 BYOK keys
- 设置 key 优先级和预算
- 配置本地模型 endpoint
- 查看调用记录

5.5 Local Bridge

运行在用户本机或内网环境的轻量服务，负责连接本地模型：

Faucet SDK -> Faucet Local Bridge -> Ollama / vLLM / LM Studio / LAN endpoint

云端 Gateway 不直接访问用户的 localhost 或 192.168.x.x 地址。

6. 四种使用模式

6.1 模式 A：平台默认水龙头

普通用户无须填写 API key。

App SDK
-> Faucet Gateway
-> Platform provider pool


```
-> LLM provider
-> Usage Ledger
-> 用户 credits 扣款
-> 渠道收益入账
```

收益公式：

```
Retail price
- Upstream model cost
- Gateway infra cost
- Payment cost
= Gross margin

Gross margin × Channel share = Developer revenue
```

6.2 模式 B：开发者自带线路

接入软件的开发者可以填入自己的 provider keys。

```
Developer Console
-> Provider credentials
-> Gateway route policy
-> Developer key 优先
-> Platform pool fallback
```

适合已有 provider 合同、企业 credits 或专属模型部署的软件公司。

6.3 模式 C：End user BYOK

终端用户填自己的 API key。

```
App SDK
-> Faucet Gateway
-> End user BYOK key
-> Provider
-> Usage Ledger
```

BYOK 模式下，上游费用通常由用户自己的 provider 账户承担。ModelFaucet 只能收取透明展示的网关费、软件服务费或订阅费，不应暗中吃 token 差价。

6.4 模式 D：本地 / 局域网模型

用户填写：

```
http://localhost:11434/v1
http://localhost:1234/v1
http://192.168.1.20:8000/v1
```

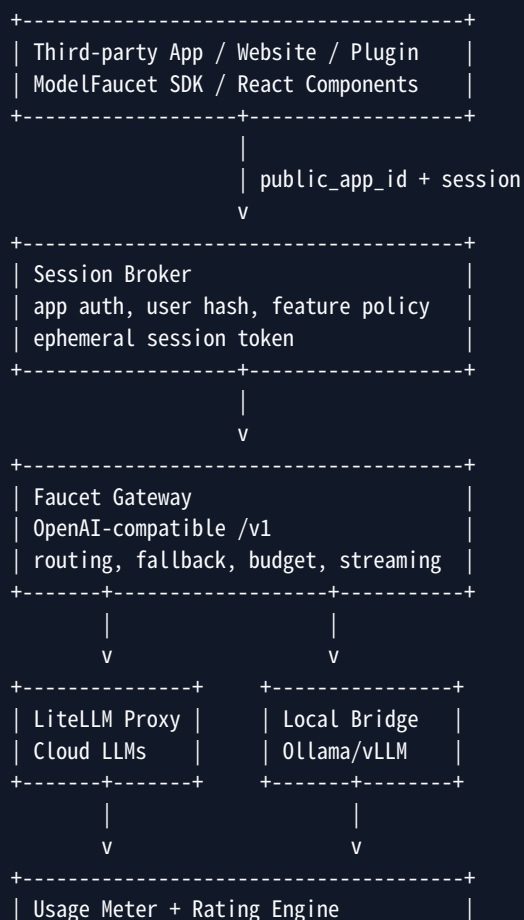
调用链：

```
App SDK
-> Faucet Local Bridge
-> Local / LAN model
-> Usage metadata 上报
```

本地模型模式通常不产生上游 token 转售利润，但可以产生：

- 本地桥接服务费
- 企业版管理费
- SDK/控制台订阅费
- 私有部署服务费

7. 总体架构



```

| token count, cost, price, margin |
+-----+-----+
|
| v
+-----+-----+
| Ledger + Billing + Settlement |
| wallets, revenue share, payout |
+-----+-----+

```

8. 核心模块

8.1 Session Broker

Session Broker 的任务是把“不可信客户端”转换成“可控短期会话”。

输入：

```

{
  "public_app_id": "app_pub_123",
  "external_user_id": "user_456",
  "feature_key": "customer_reply"
}

```

输出：

```

{
  "session_token": "mf_sess_xxx",
  "expires_in": 3600,
  "gateway_base_url": "https://gateway.modelfaucet.dev/v1",
  "available_modes": ["platform", "byok", "local"]
}

```

8.2 Gateway Router

路由策略：

```

- local_first
- byok_first
- developer_key_first
- platform_pool
- cheapest_sufficient
- quality_first
- latency_first
- privacy_first

```

路由伪代码：

```

async function selectRoute(ctx) {
  if (ctx.policy.privacy === "local_only") {
    return await requireLocalRoute(ctx);
  }

  if (ctx.userPreference === "local" && await localBridgeHealthy(ctx)) {
    return route("local");
  }

  if (ctx.userPreference === "byok" && await byokHealthy(ctx)) {
    return route("end_user_byok");
  }

  const developerCredential = await findDeveloperCredential(ctx);
  if (developerCredential) return route("developer_key", developerCredential);

  const platformCredential = await findPlatformCredential(ctx);
  if (platformCredential) return route("platform_pool", platformCredential);

  throw new Error("No available model route");
}

```

8.3 Key Vault

密钥分四类：

platform_keys	ModelFaucet 平台默认线路
operator_keys	自托管运营方线路
developer_keys	接入软件开发者自己的线路
end_user_byok	终端用户自己的 keys

密钥必须加密保存，并支持：

- key 验证
- key 禁用
- key 轮换
- key 预算
- key 使用范围
- audit log

8.4 Usage Ledger

每次调用都生成 usage event：

```

{
  "request_id": "req_abc",
  "app_id": "app_123",
  "developer_id": "dev_123",
  "end_user_id": "usr_123",
  "feature_key": "customer_reply",
  "route_mode": "platform",

```

```

"provider": "openrouter",
"model": "anthropic/claude-sonnet-4.5",
"input_tokens": 1200,
"output_tokens": 430,
"upstream_cost_usd": 0.0042,
"retail_price_usd": 0.0060,
"gross_margin_usd": 0.0018,
"channel_revenue_usd": 0.00072,
"platform_revenue_usd": 0.00108
}

```

8.5 Rating Engine

Rating Engine 把 usage event 转换为财务结果。

```

platform route:
  retail_price = upstream_cost × (1 + markup_percent)
  gross_margin = retail_price - upstream_cost
  developer_revenue = gross_margin × channel_share

byok route:
  upstream_cost = 0 for ModelFaucet
  retail_price = explicit gateway fee or subscription quota

local route:
  upstream_cost = 0 for ModelFaucet
  retail_price = explicit local bridge / enterprise service fee

```

8.6 Settlement Engine

内部钱包：

```

end_user_wallet
provider_cost_account
developer_wallet
platform_wallet

```

每次调用写双录账：

end_user_wallet	debit	retail_price
provider_cost	credit	upstream_cost
developer_wallet	credit	channel_revenue
platform_wallet	credit	platform_revenue

9. 商业与分账模型

9.1 平台默认模式

平台承担 provider 成本，用户购买 credits，平台与渠道按毛利分成。

示例：

```
上游成本: $0.0100
用户扣费: $0.0140
毛利:      $0.0040
渠道分成: 40% = $0.0016
平台收入: 60% = $0.0024
```

9.2 开发者自带线路模式

开发者自己承担 provider 成本，可选择：

- ModelFaucet 只收网关费
- ModelFaucet 参与 markup 分成
- 开发者自定义零售价格

9.3 BYOK 模式

用户自带 key 时，不应把用户自己的 provider 账单再伪装成平台转售收入。合理收费方式：

- 明确的 gateway fee
- 高级功能订阅费
- 团队管理费
- 审计/日志/合规费用

9.4 本地模型模式

本地模型没有上游 token 成本，商业化可以来自：

- 企业本地部署 license
- Local Bridge Pro
- 管理控制台
- 审计与安全策略
- 私有化技术服务

10. 安全、合规与隐私边界

10.1 不在客户端暴露真实 provider key

客户端只能拿到短期 session token 或 virtual key。真实 provider key 必须留在服务端 vault、KMS 或自托管密钥管理系统中。

10.2 不让云端访问用户内网

云端 Gateway 不直接请求：

```
localhost
127.0.0.1
10.0.0.0/8
172.16.0.0/12
192.168.0.0/16
```

本地和局域网模型必须通过 Local Bridge 调用，避免 SSRF 与内网探测风险。

10.3 用户数据最小化

ModelFaucet 不应该默认保存完整 prompt 和 completion。默认保存：

```
- request_id
- token usage
- route_mode
- model
- cost fields
- feature_key
- hashed user id
```

可选保存：

```
- prompt hash
- redacted prompt
- developer opted-in debug log
```

10.4 PII 策略

Feature Manifest 可以声明：

```
privacy_policy:
  pii: redact_before_cloud
  sensitive_fields:
    - customer_email
```

```
- phone_number
- address
fallback: local_only
```

10.5 合规提示

在公开发布前，需要律师确认：

- 各上游模型 provider 是否允许转售或白标使用
- BYOK 下用户费用和平台费用的披露方式
- 不同地区消费税、VAT、发票要求
- 数据处理协议 DPA
- 用户内容是否会被上游模型用于训练

11. 技术实施方案

11.1 推荐技术栈

Backend API:	TypeScript + Fastify 或 NestJS
Gateway wrapper:	TypeScript Fastify, 必要时迁 Go/Rust
Provider router:	LiteLLM Proxy 起步
Database:	PostgreSQL
Cache / limit:	Redis
Queue:	BullMQ 或 Temporal
Dashboard:	Next.js
SDK:	TypeScript
React components:	React + headless UI
Local Bridge:	Go 或 Rust
Observability:	OpenTelemetry + Prometheus + Grafana
Payments:	Stripe Billing + Stripe Connect 起步

11.2 Monorepo 结构

```
modelfaucet/
  apps/
    api/
    gateway/
    dashboard/
  packages/
    sdk-js/
    react/
    shared/
  services/
    local-bridge/
    rating-worker/
    settlement-worker/
  infra/
```



```
db/  
docker/  
terraform/  
docs/  
  WHITEPAPER.md  
  CONSTRUCTION.md  
  API_SPEC.md  
  SECURITY.md
```

11.3 MVP 数据库表

核心表：

```
developers  
apps  
app_features  
end_users  
provider_credentials  
virtual_sessions  
usage_events  
wallets  
ledger_entries  
payouts
```

11.4 MVP API

必须实现：

```
POST /v1/sessions  
POST /v1/chat/completions  
POST /v1/user/provider-keys  
GET /v1/user/provider-keys  
DELETE /v1/user/provider-keys/:id  
POST /v1/local/endpoints  
GET /v1/apps/:id/usage  
GET /v1/developers/:id/revenue
```

12. MVP 路线图

阶段 0：规格冻结

交付：

```
- README  
- 白皮书  
- 施工文档  
- API spec
```

- DB schema
- docker-compose
- Codex task list

阶段 1：最小闭环

目标：

- 第三方 demo app 接入 SDK
- > 用户不用 API key
- > 调用云端 LLM
- > 自动扣 credits
- > 自动计算渠道收益

验收：

- 创建 developer
- 创建 app
- 获取 public_app_id
- SDK 获取 session
- Gateway 调用 LiteLLM
- usage_events 写入
- ledger_entries 写入
- dashboard 展示收益

阶段 2：BYOK

目标：

- end user 填自己的 provider key
- > 系统验证 key
- > 后续请求优先 BYOK
- > usage 记录但不做平台 token 转售利润

阶段 3：Local Bridge

目标：

- 用户运行 Ollama / vLLM / LM Studio
- > SDK 连接 Local Bridge
- > 模型请求在本地完成
- > usage metadata 回传

阶段 4：开发者自带线路与分账

目标：

开发者配置 provider keys
-> app 按策略优先开发者线路
-> 平台 fallback
-> 开发者收益 dashboard
-> payout 流程

阶段 5：开源发布

目标：

- GitHub 仓库公开
- Apache-2.0 / AGPLv3 许可证组合
- Docker Compose 一键启动
- 文档站
- 示例插件
- 第一批垂直场景 templates

13. 开源策略

建议许可证：

SDK:	Apache-2.0 或 MIT
React components:	Apache-2.0 或 MIT
Gateway core:	AGPLv3 或 Apache-2.0
Local Bridge:	Apache-2.0
Docs:	CC-BY-4.0
Hosted service:	商业托管

如果目标是最大化接入应用数量，SDK 应采用宽松许可证。如果担心云平台被直接复制，可将 Gateway / Console 采用 AGPLv3，托管版商业化。

14. 风险与应对

14.1 API key 泄露风险

应对：

- 不在客户端存真实 key
- vault/KMS 加密
- key scope 限制
- budget cap

- anomaly detection
- audit log

14.2 上游 provider 政策风险

应对：

- 明确区分平台转售、开发者自带线路、end user BYOK
- 每个 provider 建立 terms matrix
- 允许切换 OpenRouter/Cloudflare/Vercel/自建 LiteLLM

14.3 小额高频结算复杂

应对：

- 请求时只写内部 ledger
- 用户充值走批量支付
- 开发者提现走周期性 payout

14.4 本地模型质量不稳定

应对：

- Local Bridge health check
- model capability registry
- fallback policy
- feature-level minimum capability

14.5 渠道作弊和薅羊毛

应对：

- app-level risk score
- user-level quota
- free credits abuse detection
- payout hold period
- request fingerprint

15. 参考资料

以下资料用于验证可行性与行业实现路径。正式发布前应再次复核各 provider 的最新条款。

1. LiteLLM Documentation - Self-hosted LLM Gateway, virtual keys, spend tracking
<https://docs.litellm.ai/docs/>
2. LiteLLM Virtual Keys
https://docs.litellm.ai/docs/proxy/virtual_keys
3. LiteLLM Spend Tracking
https://docs.litellm.ai/docs/proxy/cost_tracking
4. OpenAI Best Practices for API Key Safety
<https://help.openai.com/en/articles/5112595-best-practices-for-api-key-safety>
5. OpenAI API Production Best Practices
<https://developers.openai.com/api/docs/guides/production-best-practices>
6. Stripe Billing for LLM tokens
<https://docs.stripe.com/billing/token-billing>
7. Stripe Connect Destination Charges
<https://docs.stripe.com/connect/destination-charges>
8. Cloudflare AI Gateway BYOK
<https://developers.cloudflare.com/ai-gateway/configuration/bring-your-own-keys/>
9. Vercel AI Gateway BYOK
<https://vercel.com/docs/ai-gateway/authentication-and-byok/byok>
10. OpenRouter Management API Keys
<https://openrouter.ai/docs/guides/overview/auth/management-api-keys>
11. Ollama OpenAI Compatibility
<https://docs.ollama.com/api/openai-compatibility>
12. vLLM OpenAI-Compatible Server
https://docs.vllm.ai/en/latest/getting_started/quickstart/

结语

ModelFaucet 的核心不是“帮开发者接一个 AI 聊天框”，而是让 AI 能力、token 成本、用户体验、渠道收益在同一个协议层里自动流动。

当任意软件可以用几行代码接入 AI，当小白用户无须填写 API key，当高级用户可以 BYOK，当隐私用户可以本地模型，当开发者能自动获得渠道收益，AI 才真正进入垂直软件的最后一公里。

ModelFaucet = Open-source LLM Gateway + Embeddable Vertical AI SDK + BYOK + Local Bridge + Usage Ledger + Automatic Revenue Sharing.